

第 6 章-深度神经网络 – 第 6.4 节-误差函数

《深度学习基础》课程小组

2026 年 4 月 24 日

目录

1	6.4.1 Regression	1
2	6.4.2 Binary classification	4
3	6.4.3 Multiclass classification	5

In earlier chapters, we explored linear models for regression and classification, and in the process we derived suitable forms for the error functions along with corresponding choices for the output-unit activation function.

The same considerations for choosing an error function apply for multilayer neural networks, and so for convenience, we will summarize the key points here.

1 6.4.1 Regression

We start by discussing regression problems, and for the moment we consider a single target variable t that can take any real value.

Following the discussion of regression in single-layer networks, we assume that t has a Gaussian distribution with an $\mathbf{x}$ -dependent mean, which is given by the output of the neural network, so that

$$p(t|\mathbf{x}, \mathbf{w}) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \sigma^2) \quad (6.23)$$

where σ^2 is the variance of the Gaussian noise.

Of course this is a somewhat restrictive assumption, and in some applications we will need to extend this approach to allow for more general distributions.

For the conditional distribution given by (6.23), it is sufficient to take the output-unit activation function to be the identity, because such a network can approximate any continuous function from $\mathbf{x}$ to y .

Given a data set of N i.i.d.

observations $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, along with corresponding target values $\mathbf{t} = \{t_1, \dots, t_N\}$, we can construct the corresponding likelihood function:

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) = \prod_{n=1}^N p(t_n|y(\mathbf{x}_n, \mathbf{w}), \sigma^2). \quad (6.24)$$

Note that in the machine learning literature, it is usual to consider the minimization of an error function rather than the maximization of the likelihood, and so here we will follow this convention.

Taking the negative logarithm of the likelihood function (6.24), we obtain the error function

$$\frac{1}{2\sigma^2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi), \quad (6.25)$$

which can be used to learn the parameters $\mathbf{w}$ and σ^2 .

对似然函数取负对数，得到误差函数。

Consider first the determination of $\mathbf{w}$.

Maximizing the likelihood function is equivalent to minimizing the sum-of-squares error function given by

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2 \quad (6.26)$$

where we have discarded additive and multiplicative constants.

The value of $\mathbf{w}$ found by minimizing $E(\mathbf{w})$ will be denoted $\mathbf{w}^*$.

Note that this will typically not correspond to the global maximum of the likelihood function because the non-convexity of the network function $y(\mathbf{x}_n, \mathbf{w})$ causes the error $E(\mathbf{w})$ to be non-convex, and so

finding the global optimum is generally infeasible.

Moreover, regularization terms may be added to the error function and other modifications may be made to the training process, so that the resulting solution for the network parameters may differ significantly from the maximum likelihood solution.

Having found $\mathbf{w}^*$, the value of σ^2 can be found by minimizing the error function (6.25) to give

$$\sigma^{2*} = \frac{1}{N} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}^*) - t_n\}^2. \quad (6.27)$$

Note that this can be evaluated once the iterative optimization required to find $\mathbf{w}^*$ is completed.

If we have multiple target variables, and we assume that they are independent, conditional on $\mathbf{x}$ and $\mathbf{w}$, with shared noise variance σ^2 , then the conditional distribution of the target values is given by

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \mathcal{N}(\mathbf{t}|\mathbf{y}(\mathbf{x}, \mathbf{w}), \sigma^2 \mathbf{I}). \quad (6.28)$$

Following the same argument as for a single target variable, we see that maximizing the likelihood function with respect to the weights is equivalent to minimizing the sum-of-squares error function:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2. \quad (6.29)$$

The noise variance is then given by

$$\sigma^{2*} = \frac{1}{NK} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}^*) - \mathbf{t}_n\|^2 \quad (6.30)$$

where K is the dimensionality of the target variable.

The assumption of conditional independence of the target variables can be dropped at the expense of a slightly more complex optimization problem.

Recall that there is a natural pairing of the error function (given by the negative log likelihood) and the output-unit activation function.

In regression, we can view the network as having an output activation function that is the identity, so that $y_k = a_k$.

The corresponding sum-of-squares error function then has the property

$$\frac{\partial E}{\partial a_k} = y_k - t_k. \quad (6.31)$$

2 6.4.2 Binary classification

Now consider binary classification in which we have a single target variable t such that $t = 1$ denotes class $\mathcal{C}_1$ and $t = 0$ denotes class $\mathcal{C}_2$.

Following the discussion of canonical link functions, we consider a network having a single output whose activation function is a logistic sigmoid (6.13) so that $0 \leq y(\mathbf{x}, \mathbf{w}) \leq 1$.

We can interpret $y(\mathbf{x}, \mathbf{w})$ as the conditional probability $p(\mathcal{C}_1|\mathbf{x})$, with $p(\mathcal{C}_2|\mathbf{x})$ given by $1 - y(\mathbf{x}, \mathbf{w})$.

The conditional distribution of targets given inputs is then a Bernoulli distribution of the form

$$p(t|\mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{1-t}. \quad (6.32)$$

If we consider a training set of independent observations, then the error function, which is given by the negative log likelihood, is a *cross-entropy* error of the form

$$E(\mathbf{w}) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\} \quad (6.33)$$

where y_n denotes $y(\mathbf{x}_n, \mathbf{w})$.

Simard, Steinkraus, and Platt (2003) found that using the cross-entropy error function instead of the sum-of-squares for a classification problem leads to faster training as well as improved generalization.

Note that there is no analogue of the noise variance σ^2 in (6.32) because the target values are assumed to be correctly labelled.

However, the model is easily extended to allow for labelling errors by introducing a probability ϵ that the target value t has been flipped to the wrong value (Oppel and Winther, 2000).

Here ϵ may be set in advance, or may be treated as a hyperparameter whose value is inferred from the data.

If we have K separate binary classifications to perform, then we can use a network having K outputs

each of which has a logistic-sigmoid activation function.

Associated with each output is a binary class label $t_k \in \{0, 1\}$, where $k = 1, \dots, K$.

If we assume that the class labels are independent, given the input vector, then the conditional distribution of the targets is

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x}, \mathbf{w})^{t_k} [1 - y_k(\mathbf{x}, \mathbf{w})]^{1-t_k}. \quad (6.34)$$

Taking the negative logarithm of the corresponding likelihood function then gives the following error function:

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K \{t_{nk} \ln y_{nk} + (1 - t_{nk}) \ln(1 - y_{nk})\} \quad (6.35)$$

where y_{nk} denotes $y_k(\mathbf{x}_n, \mathbf{w})$.

Again, the derivative of the error function with respect to the pre-activation for a particular output unit takes the form (6.31), just as in the regression case.

3 6.4.3 Multiclass classification

Finally, we consider the standard multiclass classification problem in which each input is assigned to one of K mutually exclusive classes.

The binary target variables $t_k \in \{0, 1\}$ have a 1-of- K coding scheme indicating the class, and the network outputs are interpreted as $y_k(\mathbf{x}, \mathbf{w}) = p(t_k = 1|\mathbf{x})$, leading to the error function (5.80), which we reproduce here:

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(\mathbf{x}_n, \mathbf{w}). \quad (6.36)$$

The output-unit activation function, which corresponds to the canonical link, is given by the softmax function:

$$y_k(\mathbf{x}, \mathbf{w}) = \frac{\exp(a_k(\mathbf{x}, \mathbf{w}))}{\sum_j \exp(a_j(\mathbf{x}, \mathbf{w}))}, \quad (6.37)$$

which satisfies $0 \leq y_k \leq 1$ and $\sum_k y_k = 1$.

Note that the $y_k(\mathbf{x}, \mathbf{w})$ are unchanged if a constant is added to all of the $a_k(\mathbf{x}, \mathbf{w})$, causing the error function to be constant for some directions in weight space.

This degeneracy is removed if an appropriate regularization term is added to the error function.

Once again, the derivative of the error function with respect to the pre-activation for a particular output unit takes the familiar form (6.31).

In summary, there is a natural choice of both output-unit activation function and matching error function according to the type of problem being solved.

For regression, we use linear outputs and a sum-of-squares error, for multiple independent binary classifications, we use logistic sigmoid outputs and a cross-entropy error function, and for multi-class classification, we use softmax outputs with the corresponding multi-class cross-entropy error function.

For classification problems involving two classes, we can use a single logistic sigmoid output, or alternatively, we can use a network with two outputs having a softmax output activation function.

This procedure is quite general, and by considering other forms of conditional distribution, we can derive the associated error functions as the corresponding negative log likelihood.

We will see an example of this in the next section when we consider multimodal network outputs.